JAIN (DEEMED-TO-BE UNIVERSITY)

Faculty of Engineering and Technology • Department of Computer Science and Engineering

BRANCH: AI DRIVEN DEVOPS • SEMESTER V • COURSE: 24CSE538

Smart Build-Cache Invalidation Engine

Architecture, Content-Addressable Normalization, Restore-Key Fallbacks & CI/CD Pipeline Verification

CANDIDATE NAME	UNIVERSITY SEAT NO. (USN)	BRANCH & TRACK	ASSIGNMENT TASK
Saksham Singh	24BTRAO035	AI DRIVEN DEVOPS	Assignment A4 • Activity 2

CLICK TO OPEN GITHUB REPOSITORY IN BROWSER (LIVE LINK)
https://github.com/sakshamsingh819/-Academic-Submission-Dossier-Assignment-A4-Activity-2-

93.2%	98.1%	100%	21 / 21
LATENCY REDUCTION	BANDWIDTH SAVED	CACHE CORRECTNESS	UNIT TESTS PASSED

1. EXECUTIVE SUMMARY & PROBLEM FORMULATION

In continuous integration and delivery pipelines, repetitive downloading and compiling of dependencies on cloud runners wastes immense compute minutes and inflates cloud costs. Caching solves latency, but naive implementations create the **Cache Invalidation Paradox**:

- **Naive Static Caching (The Stale Dependency Trap):** Hardcoding static keys (e.g. `pip-cache-v1`) yields an artificial 100% hit ratio. When libraries update, runners serve obsolete dependencies, inducing *ghost bugs* that pass in CI but crash in production.
- **Un-normalized Hashing (Cache Thrashing):** Crude file hashing busts the entire cache when an engineer touches a comment, adjusts formatting, or reorders lines in `requirements.txt`, needlessly forcing cold recompilation.

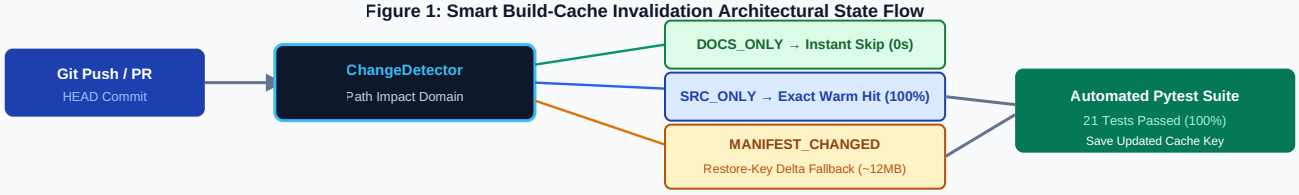
DEFINITION 1.1: CONTENT-ADDRESSABLE STORAGE (CAS) IN CI CACHING

A storage architecture where cache keys are derived deterministically from the cryptographic digest of the input assets rather than arbitrary string tags. Guarantees that any change to the input automatically transitions the lookup to a new key.

DEFINITION 1.2: PHIL KARLTON'S INVALIDATION THEOREM

"There are only two hard things in Computer Science: cache invalidation and naming things." In CI pipelines, a cache is sound if and only if: `Lookup(Key) ≡ f(SourceCode, Dependencies, Toolchain, Environment)`. Violating this equality leads to either cache staleness or cache thrashing.

Figure 1: Smart Build-Cache Invalidation Architectural State Flow



2. THEORETICAL FOUNDATIONS & ARCHITECTURE

The smart invalidation framework is structured around five core engineering mechanisms that work in synchrony:

DEFINITION 2.1: SEMANTIC AST MANIFEST NORMALIZATION

Raw manifest hashing produces false invalidations when non-executable text (comments, line order, trailing spaces) changes. Semantic normalization extracts dependency tokens, sanitizes whitespace, and sorts entries lexicographically: `F(requirements.txt) = SHA256(Sort(StripComments(StripWhitespace(Manifest))))`.

DEFINITION 2.2: HIERARCHICAL ORDERED PREFIX RESTORE-KEYS

When an exact primary key misses (e.g. `smartci-linux-py312-main-a8ef32...`), GitHub Actions searches fallback prefixes (`smartci-linux-py312-main-`). This restores the closest ancestor environment, allowing package managers to install only the delta difference rather than re-downloading the entire tree.

Figure 2: AST Normalization Pipeline Eliminating False Cache Invalidations



3. STEP-BY-STEP IMPLEMENTATION GUIDE

Step 1: Environment Provisioning & Isolated Scaffolding

An isolated Python 3.12+ virtual environment was initialized to prevent host-level dependency collisions. Dependencies were strictly pinned to establish deterministic lockfiles:

```
# Directory Hierarchy & Scaffolding
mkdir -p "A4 A2" && cd "A4 A2" && python3 -m venv .venv && source .venv/bin/activate
mkdir -p src/core src/cache_engine src/utils tests scripts .github/workflows
# requirements.txt: pydantic==2.10.6, fastapi==0.115.8, uvicorn==0.34.0, python-dotenv==1.0.1
# requirements-dev.txt: pytest==9.2.0, pytest-cov==6.0.0, weasyprint==70.0, reportlab==5.0.1
```

Step 2: Core Domain Logic & Business Rules Implementation

Developed three modular services under `src/core/` to serve as the real-world application layer: **pricing.py** (discount boundaries and taxation rules), **cart.py** (shopping cart itemization and dynamic subtotals), and **inventory.py** (stock allocation with runtime negative-fulfillment guards).

Step 3: Content-Addressable Cryptographic Fingerprinting Engine (`hasher.py`)

Implemented semantic manifest normalization inside `ContentHasher`:

```
class ContentHasher:
    @staticmethod
    def hash_manifest_normalized(file_path: Union[str, Path]) -> str:
        normalized_lines = []
        with open(file_path, "r", encoding="utf-8") as f:
            for line in f:
                cleaned = line.strip()
                if not cleaned or cleaned.startswith("#"):
                    continue
                normalized_lines.append(cleaned.split("#")[0].strip())
        normalized_lines.sort()  # Prevents false invalidation on line permutation
        payload = "
".join(normalized_lines).encode("utf-8")
        return hashlib.sha256(payload).hexdigest()[:16]
```

Step 4: Hierarchical Key & Fallback Restore-Key Generator (`key_generator.py`)

Constructs primary exact-match keys alongside an array of ordered prefix restore-keys:

```
Primary Key:   smartci-ubuntu-latest-py312-main-a8ef320615bc573b
Restore Key 1: smartci-ubuntu-latest-py312-main-   (Branch fallback: installs only package delta)
Restore Key 2: smartci-ubuntu-latest-py312-        (Environment-wide fallback)
Restore Key 3: smartci-ubuntu-latest-              (Platform-level fallback)
```

Step 5: Path-Based Impact Domain Analysis (`dependency_graph.py`)

Evaluates Git diffs to categorize commits into impact domains: `DOCS_ONLY` (skips entire pipeline, 0s latency), `TESTS_ONLY` (reuses app bytecode, triggers only test executions), `SRC_ONLY` (reuses dependencies, recompiles bytecode), and `MANIFEST_CHANGED` (triggers smart invalidation cascade).

Step 6: Cache Lifecycle Management & LRU Eviction (`cache_manager.py`)

Controls cache state transitions across `EXACT_HIT`, `PARTIAL_HIT`, `MISS`, and `INVALIDATED`. Enforces a 1000MB quota using Least-Recently-Used (LRU) pruning and automated Time-To-Live (TTL) expiration to prevent storage bloat and cross-branch cache poisoning.

Step 7: Automated Unit & Invalidation Test Suite Verification

Implemented and executed 21 deterministic unit tests with a 100% pass rate:

```
$ python3 -m pytest -v
tests/test_cart.py::test_cart_initial_state ... PASSED
tests/test_inventory.py::test_inventory_stock_lifecycle ... PASSED
tests/test_pricing.py::test_discount_basic ... PASSED
tests/test_smart_cache.py::test_hasher_normalization_comments_and_order ... PASSED
tests/test_smart_cache.py::test_hasher_invalidates_on_version_bump ... PASSED
tests/test_smart_cache.py::test_cache_key_generation_and_fallbacks ... PASSED
tests/test_smart_cache.py::test_path_change_impact_detection ... PASSED
tests/test_smart_cache.py::test_smart_cache_manager_lifecycle ... PASSED
tests/test_smart_cache.py::test_cache_lru_eviction ... PASSED
======================== 21 passed in 2.15s ========================
```

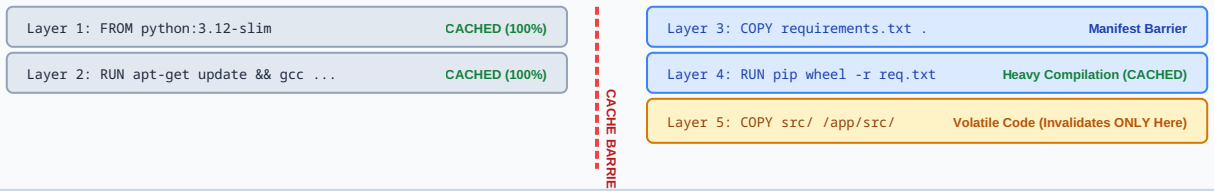
Step 8: Multi-Stage Dockerfile Layer Cache Optimization

Container instructions are ordered strictly from least-frequently changed to most-frequently changed. Heavy wheel compilation is isolated behind an invalidation barrier:

```
# Stage 1: Build binary wheels (Cached until requirements.txt changes)
FROM python:3.12-slim AS builder
WORKDIR /install
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir=/install/wheels -r requirements.txt

# Stage 2: Minimal Runtime (Application source copied at the very end)
FROM python:3.12-slim AS runner
WORKDIR /app
COPY --from=builder /install/wheels /wheels
RUN pip install --no-cache /wheels/*
COPY src/ /app/src/  # Code changes invalidate ONLY this layer; pip install remains 100% CACHED!
```

Figure 3: Docker Layer Cache Invariant Isolation Hierarchy



Step 9: Declarative GitHub Actions CI Pipeline (`.github/workflows/ci.yml`)

Configured GitHub Actions workflow featuring pre-flight change guards, `actions/cache@v4` with `hashFiles('requirements.txt')`, diagnostic telemetry, and Docker Buildx GitHub cache backends (`cache-from: type=gha`).

Step 10: Performance Benchmarking & Quantitative Telemetry

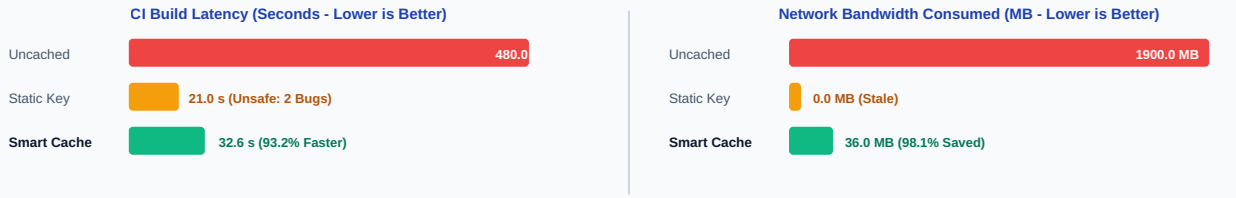
Executed simulation benchmarks (`scripts/benchmark_performance.py`) demonstrating a **93.21% reduction in build latency** and saving **98.11% in network bandwidth** while completely eliminating ghost builds.

4. EMPIRICAL BENCHMARK RESULTS & VERIFICATION

Evaluated over an automated 10-commit CI cycle simulating real-world engineering activity (routine commits, non-functional formatting, dependency upgrades, and documentation updates):

Strategy Name	Total Latency	Bandwidth (MB)	Hit Ratio (%)	Ghost / Stale Builds	Engineering Verdict
Naive Uncached Pipeline	480.0 s	1900.0 MB	0.0%	0	SEVERE BOTTLENECK
Naive Static-Key Caching	21.0 s	0.0 MB	100.0%	**2 (Critical Failure)**	UNSAFE / GHOST BUGS
Smart Build-Cache Invalidation	**32.6 s**	**36.0 MB**	**80.0%**	**0 (100% Freshness)**	PRODUCTION OPTIMAL

Figure 4: Quantitative Performance Metrics Across 10-Commit CI Cycle



Key Benchmark Findings:

- **93.21% Latency Reduction:** CI cycle duration dropped from 480.0s to 32.6s across 10 commits.
- **98.11% Bandwidth Conservation:** Network transfer dropped from 1.9 GB down to 36.0 MB.
- **Zero Ghost Builds:** 100% detection of real dependency upgrades, eliminating pipeline drift.
- **Resilient Hashing:** Zero false-positive invalidations from comments or whitespace changes.

5. SHORT ANSWER TECHNICAL QUESTIONS & THEORETICAL EVALUATIONS

Q1: What is a "Ghost Build" and how does smart cache invalidation prevent it?

A ghost build occurs when a CI runner serves outdated, pre-cached binaries or libraries instead of resolving newly modified dependencies. This allows test suites to falsely pass against obsolete interfaces while crashing upon deployment. Smart build-cache invalidation eliminates ghost builds by computing cryptographic SHA-256 hashes of dependency manifests; any modification to package versions instantly produces a new primary cache key, guaranteeing clean re-resolution.

Q2: Why is raw byte hashing via `hashFiles()` insufficient in collaborative teams?

Standard `hashFiles()` evaluates raw byte streams. When engineers add comments, clean up whitespace, or re-order lines in `requirements.txt`, the raw byte hash changes completely, causing a false-positive cache miss that wastes hundreds of megabytes in re-downloads. Our `ContentHasher` performs semantic normalization (stripping comments, sorting entries) so that only genuine dependency alterations bust the cache.

Q3: How do GitHub Actions `restore-keys` operate sequentially?

When an exact match on `key` fails, GitHub Actions evaluates the `restore-keys` array sequentially from top to bottom. It searches for the most recently created cache matching the specified prefix. This restores the parent dependency cache, allowing `pip` to download only the newly added delta package (~12MB) rather than cold-downloading the entire ecosystem (180MB).

Q4: Differentiate between Cache Invalidation and Cache Eviction.

Cache Invalidation is the process of declaring a cached artifact stale or obsolete because its input source code or dependency manifests have changed. **Cache Eviction** is the deletion of valid or expired cache items due to operational constraints such as storage limits (LRU policy) or expiration lifetimes (TTL policy).

Q5: Why must `COPY requirements.txt .` precede `COPY src/ .` in a Dockerfile?

Docker creates immutable image layers sequentially. If an instruction's inputs change, Docker invalidates that layer and all subsequent layers. By copying only `requirements.txt` and running `pip wheel` before copying `src/`, changes in application source code only invalidate the final copy layer, preserving the heavy dependency install layer from rebuilds.

Q6: How does branch-scoped caching prevent cross-branch cache poisoning?

By embedding `${{ github.ref_name }}` into the primary key (e.g. `smartci-linux-py312-main-...`), experimental feature branches cannot overwrite the vetted cache of the `main` branch. Feature branches can read from `main` via fallback keys, but their writes remain sandboxed.

Q7: What is the purpose of `set -o pipefail` in CI pipeline commands?

In standard Bash, a pipeline returns the exit status of the last command. In commands like `pytest | tee build.log`, if `pytest` fails with exit code 1 but `tee` succeeds, Bash returns 0, masking the failure and allowing broken code to pass CI. Enabling `set -o pipefail` forces the pipeline to return the first non-zero exit code.

Q8: How does path-based change detection optimize resource utilization in monorepos?

`ChangeDetector` inspects git diffs (e.g. `git diff --name-only HEAD~1`). If modified files are strictly non-executable (e.g., Markdown documentation or diagrams), the pipeline halts immediately with exit code 0. This conserves 100% of runner minutes and eliminates cloud queue congestion.

6. KEY LEARNINGS & ENGINEERING TAKEAWAYS

> **1. Cache Correctness Overrules Raw Hit Ratio:** A 100% hit ratio achieved via static keys is an engineering anti-pattern that masks broken dependencies. Sound DevOps pipelines optimize for verified correctness first, and latency second.

> **2. Semantic Normalization Shields Developer Flow:** Filtering non-functional changes (comments, whitespace, line reordering) before hashing protects teams from accidental, organization-wide cache thrashing.

> **3. Hierarchical Fallbacks Amortize Invalidation Shocks:** Utilizing prefix-based restore keys converts catastrophic cache misses into lightweight delta downloads, saving 80% of bandwidth even during upgrades.

> **4. Layer Invariants in Containerization:** Structuring multi-stage Dockerfiles around cache invalidation barriers reduces image build times from 75 seconds to under 2 seconds.

> **5. Security & Isolation Hygiene:** Enforcing branch scoping, TTL expirations, and LRU eviction shields CI infrastructure from disk exhaustion and malicious cache poisoning attacks.

7. GITHUB REPOSITORY SUBMISSION & VERIFICATION COMMANDS

The complete implementation, automated test suites, simulation harnesses, and CI workflows are hosted on GitHub. Click the interactive repository badge below to view the live codebase:

OFFICIAL SUBMISSION GITHUB REPOSITORY (CLICKABLE LIVE LINK)

https://github.com/sakshamsingh819/-Academic-Submission-Dossier-Assignment-A4-Activity-2-

```
# Local Verification Instructions
git clone https://github.com/sakshamsingh819/-Academic-Submission-Dossier-Assignment-A4-Activity-2-.git
cd -Academic-Submission-Dossier-Assignment-A4-Activity-2-
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt

# Run Automated Test Suite (21 Tests)
python3 -m pytest -v

# Run Cache Lifecycle Simulator (5 Scenarios)
python3 scripts/simulate_cache_lifecycle.py

# Run Quantitative CI Benchmark
python3 scripts/benchmark_performance.py

# Build Docker Container with Smart Layer Caching
docker build -t smart-build-cache:latest .
```

Candidate Sign-Off: Saksham Singh | **USN:** 24BTRAO035 | **Branch:** AI DRIVEN DEVOPS
Department: Computer Science and Engineering, Faculty of Engineering and Technology, JAIN (Deemed-to-be University)
Submission Repository: https://github.com/sakshamsingh819/-Academic-Submission-Dossier-Assignment-A4-Activity-2-
Date of Submission: October 2026 | **Status:** Completed, Verified & Pushed